

Unit II: Symmetric Cryptography

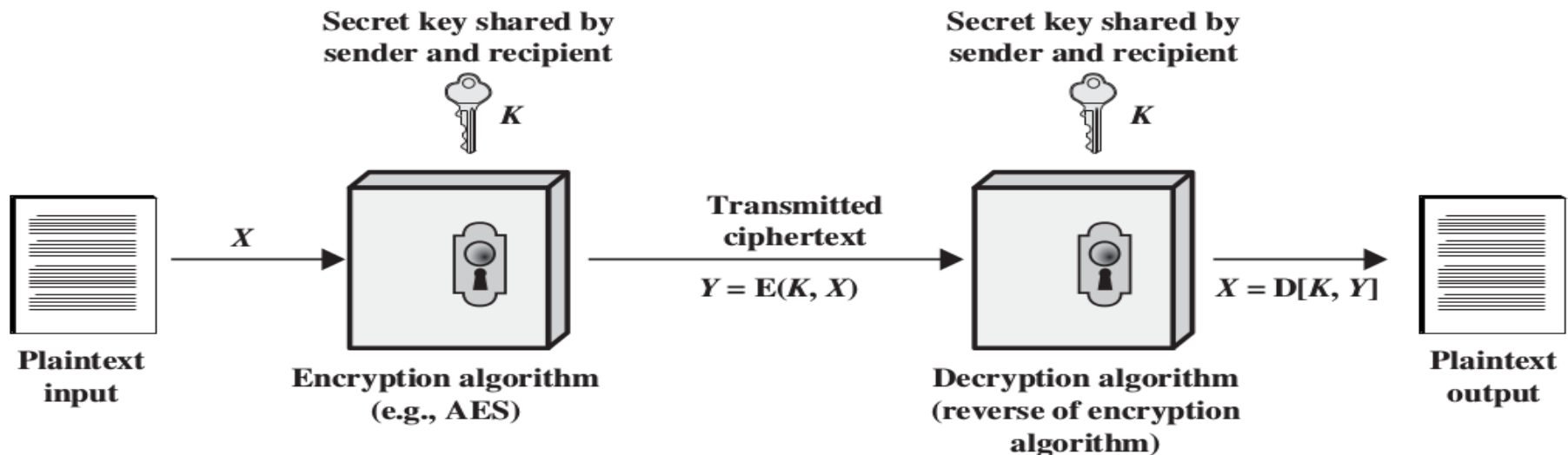
Roshan Chitrakar, PhD
Professor at NCIT
roshanchi@gmail.com

Conventional Cryptography

- ***Symmetric encryption***, also referred to as conventional encryption or single-key encryption, was the only type of encryption in use prior to the development of public-key encryption in the 1970s.
- An original message is known as the ***plaintext***, while the coded message is called the ***ciphertext***
- The process of converting from plaintext to ciphertext is known as ***enciphering*** or ***encryption***; restoring the plaintext from the ciphertext is ***deciphering*** or ***decryption***.
- Many schemes used for encryption constitute the area of study known as ***cryptography***. Such a scheme is known as a ***cryptographic system*** or a ***cipher***.
- The areas of cryptography and ***cryptanalysis*** together are called ***cryptology***

Symmetric Cipher Model

- A symmetric encryption scheme has **five (5)** ingredients.
 - **Plaintext**: This is the original intelligible message or data that is fed into the algorithm as input.
 - **Encryption algorithm**: The encryption algorithm performs various substitutions and transformations on the plaintext.
 - **Secret key**: The secret key is also input to the encryption algorithm. The key is a value independent of the plaintext and of the algorithm. The algorithm will produce a different output depending on the specific key being used at the time. The exact substitutions and transformations performed by the algorithm depend on the key.
 - **Ciphertext**: This is the scrambled message produced as output. It depends on the plaintext and the secret key. For a given message, two different keys will produce two different ciphertexts. The ciphertext is an apparently random stream of data and, as it stands, is unintelligible.
 - **Decryption algorithm**: This is essentially the encryption algorithm run in reverse. It takes the ciphertext and the secret key and produces the original plaintext.

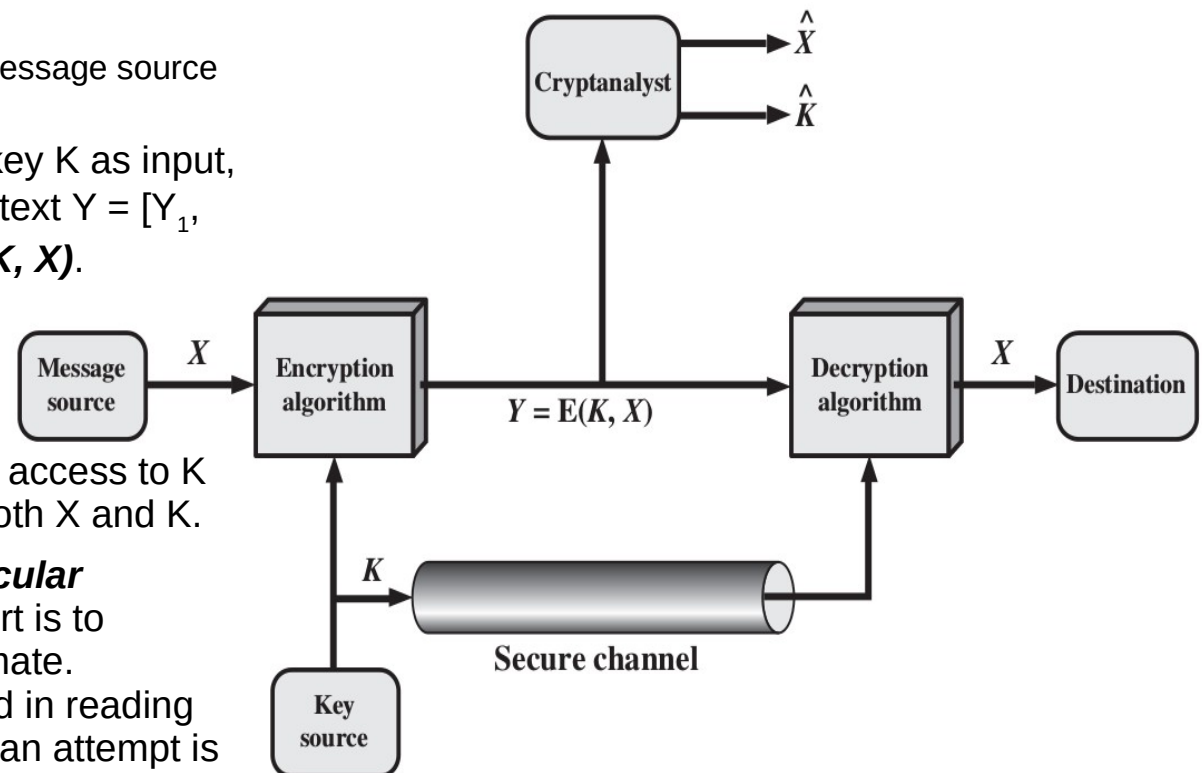


Secure Conventional Encryption

- There are ***two requirements*** for secure use of conventional encryption:
 - 1. We need a ***strong encryption algorithm***.
 - An opponent who knows the algorithm and one or more ciphertexts would be unable to decipher the ciphertext or figure out the key.
 - 2. The ***key must be secure***.
 - Sender and receiver must obtain the secret key in a secure fashion and must keep the key secure.
- We assume that it is ***impractical to decrypt*** a message on the basis of the ciphertext plus knowledge of the encryption/decryption algorithm
- we do not need to keep the algorithm secret; we need to keep only the key secret. So, the principal security problem is maintaining the secrecy of the key

Symmetric Cryptosystem

- A **source produces a message** in plaintext, $X = [X_1, X_2, \dots, X_M]$.
 - (The M elements of X are letters in some finite alphabets.)
- For encryption, **a key is generated** i.e. $K = [K_1, K_2, \dots, K_j]$.
 - (This key may either be generated at the message source or a third party could generate.)
- With the message X and the encryption key K as input, the encryption algorithm forms the ciphertext $Y = [Y_1, Y_2, \dots, Y_N]$ represented by eqn $Y = E(K, X)$.
- The intended receiver, in possession of the key, is able to invert the transformation $X = D(K, Y)$.
- An opponent, observing Y but not having access to K or X , may attempt to recover X or K or both X and K .
- If the opponent is interested in **this particular message only**, then the focus of the effort is to **recover X** by generating a plaintext estimate. However, the opponent is often interested in reading **future messages** as well, in which case an attempt is made to **recover K** by generating an estimate.



Cryptography

- Characterized along 3 independent dimensions: -
 1. The ***type of operations/algorithms*** used for transforming plaintext to ciphertext.
 - ***Substitution*** : each element in the plaintext (bit, letter, group of bits or letters) is mapped into another element,
 - ***Transposition*** : elements in the plaintext are rearranged.
 - No information should be lost (that is, all operations are reversible).
 2. The ***number of keys*** used.
 - ***Single-key***, secret-key, (symmetric or conventional encryption)
 - ***Two-keys***, or public-key, (asymmetric encryption)
 3. The ***way the plaintext is processed***.
 - ***Block cipher*** : processes the input one block of elements at a time, producing an output block for each input block
 - ***Stream cipher*** : processes the input elements continuously, producing output one element at a time

Approaches of Attack

- The objective of attacking an encryption system is ***to recover the key*** in use rather than simply to recover the plaintext of a single ciphertext
- Two general approaches to attacking a conventional encryption scheme: -
 - ***Cryptanalysis:***
 - exploits ***the characteristics of the algorithm*** to attempt to deduce a specific plaintext or to deduce the key being used.
 - some knowledge of the general characteristics of the plaintext or some sample plaintext–ciphertext pairs are analysed.
 - ***Brute-Force Attack:***
 - tries ***every possible key*** on a piece of ciphertext until an intelligible translation into plaintext is obtained.
 - On average, half of all possible keys must be tried to achieve a success.

How secure is an algorithm?

- ***Unconditionally secure***

- The ciphertext generated by the scheme ***does not contain enough information*** to determine uniquely the corresponding plaintext, ***no matter how much ciphertext*** is available.
- ***No matter how much time*** an opponent has, it is ***impossible to decrypt*** the ciphertext simply because the required information is not there.
- ***No encryption algorithm is unconditionally secure***. Therefore, all that the users of an encryption algorithm can strive for is an algorithm that meets one or both of the following criteria:
 - The ***cost of breaking the cipher*** exceeds the value of the encrypted information.
 - The ***time required to break*** the cipher exceeds the useful lifetime of the information.

- ***Computationally secure***

- Either of the afore-mentioned two criteria are met.

Attacks: How effective?

Cryptanalytic attacks: -

Cryptanalysis for symmetric encryption schemes are designed to exploit the fact that traces of structure or pattern in the ***plaintext may survive encryption*** and be discernible in the ciphertext.

Brute-force attacks: -

Key Size (bits)	Number of Alternative Keys	Time Required at 1 Decryption/ μ s	Time Required at 10^6 Decryptions/ μ s
32	$2^{32} = 4.3 \times 10^9$	$2^{31} \mu\text{s} = 35.8 \text{ minutes}$	2.15 milliseconds
56	$2^{56} = 7.2 \times 10^{16}$	$2^{55} \mu\text{s} = 1142 \text{ years}$	10.01 hours
128	$2^{128} = 3.4 \times 10^{38}$	$2^{127} \mu\text{s} = 5.4 \times 10^{24} \text{ years}$	$5.4 \times 10^{18} \text{ years}$
168	$2^{168} = 3.7 \times 10^{50}$	$2^{167} \mu\text{s} = 5.9 \times 10^{36} \text{ years}$	$5.9 \times 10^{30} \text{ years}$
26 characters (permutation)	$26! = 4 \times 10^{26}$	$2 \times 10^{26} \mu\text{s} = 6.4 \times 10^{12} \text{ years}$	$6.4 \times 10^6 \text{ years}$

56-bit key : DES; 168-bit key : Triple DES; 128-bit key : AES ; 26 char : Substⁿ codes

- 1 decryption/ μ s is the reasonable order of magnitude of today's computer.
- 10^6 decryption/ μ s can be achieved with the use of massively parallel processors.

Substitution Techniques: Caesar Cipher

- The earliest known, and the simplest, use of a substitution cipher was by Julius Caesar
 plain: meet me after the toga party
 cipher: PHHW PH DIWHU WKH WRJD SDUWB
- Replaces each letter of the alphabet with the letter standing three places further down the alphabet

- define the transformation by listing all possibilities

plain: a b c d e f g h i j k l m n o p q r s t u v w x y z
 cipher: D E F G H I J K L M N O P Q R S T U V W X Y Z A B C

- assign a numerical equivalent to each letter

a	b	c	d	e	f	g
0	1	2	3	4	5	6

	r	s	t	u	v	w	x	y	z
7	17	18	19	20	21	22	23	24	25

- the algorithm can be expressed as follows. For each plaintext letter p , substitute the ciphertext letter C

$$C = E(3, p) = (p + 3) \bmod 26$$

- A shift may be of any amount, so that the general Caesar algorithm is

$$C = E(k, p) = (p + k) \bmod 26$$

- The decryption algorithm is

$$p = D(k, C) = (C - k) \bmod 26$$

Caesar Cipher: How to break?

A brute-force cryptanalysis is performed: try all the 25 possible keys.

KEY		PHHW	PH	DIWHU	WKH	WRJD	SDUWB
1		oggv	og	chvgt	vjg	vqic	rctva
2		nffu	nf	bgufs	uif	uphb	qbsuz
3		meet	me	after	the	toga	party
4		ldds	ld	zesdq	sgd	snfz	ozqsx

Three important characteristics of this problem enabled us to use a brute-force cryptanalysis:

1. The encryption and decryption algorithms are known.
2. There are only 25 keys to try.
3. The language of the plaintext is known and easily recognizable.

What generally makes brute-force cryptanalysis impractical is the use of an algorithm that employs a large number of keys.

For example, the triple DES algorithm makes use of a 168-bit key, giving a key space of 2^{168} or greater than $3.7 * 10^{50}$ possible keys

Substitution Techniques (contd.)

Monoalphabetic Ciphers

If, instead, the “cipher” line of Caesar Cipher can be ***any permutation of the 26 alphabetic characters***, then there are $26!$ or greater than 4×10^{26} possible keys.

This is 10 orders of magnitude greater than the key space for DES and would seem to eliminate brute-force techniques for cryptanalysis

Ciphertext: -

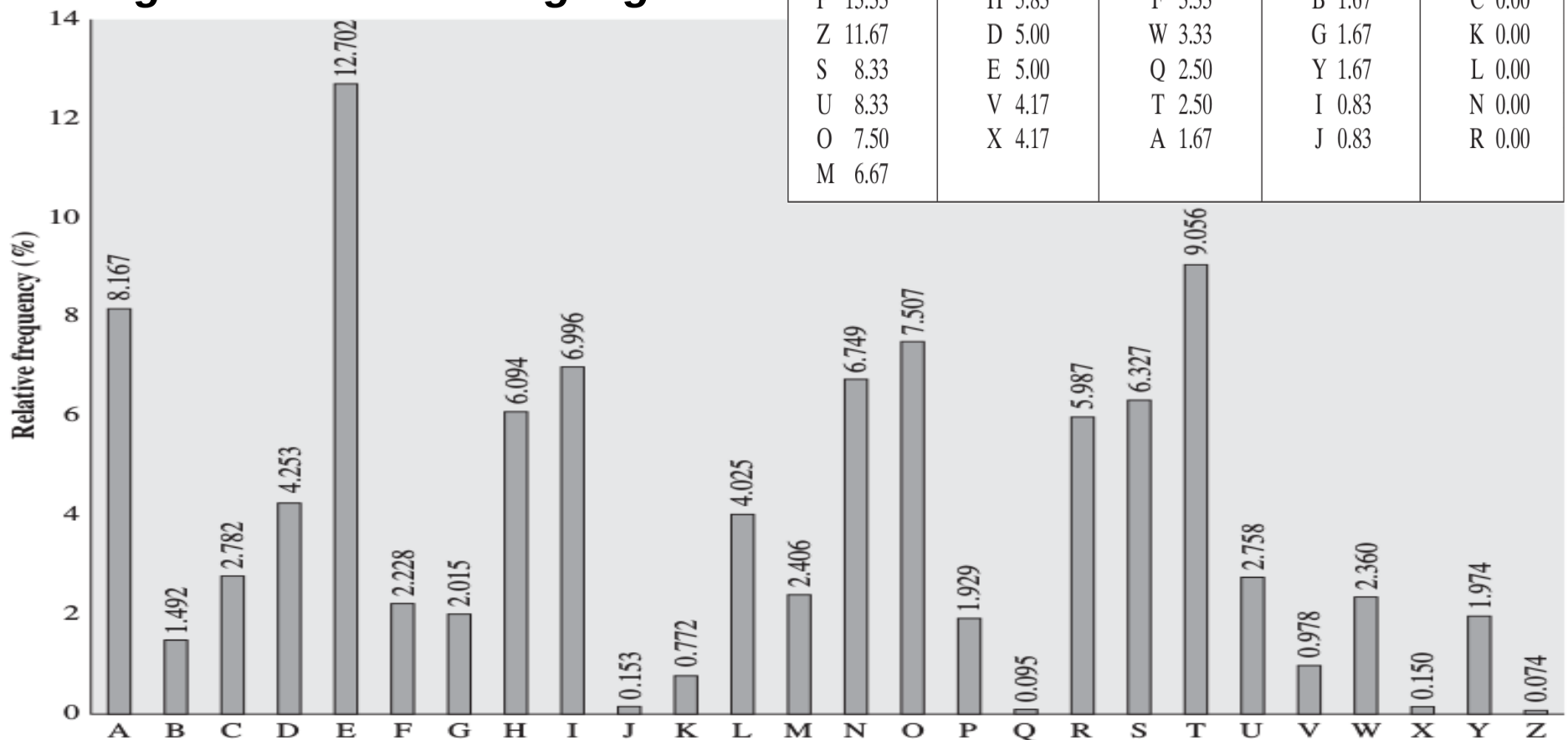
```
UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFPESXUDBMETSXAI Z  
VUEPHZHMDZSHZOWSFPAPDTSVPQUZWYMXUZUHSX  
EPYEPOPDZSZUFPOMBZWPFUPZHMDJUDTMOHMQ
```

Plaintext: -

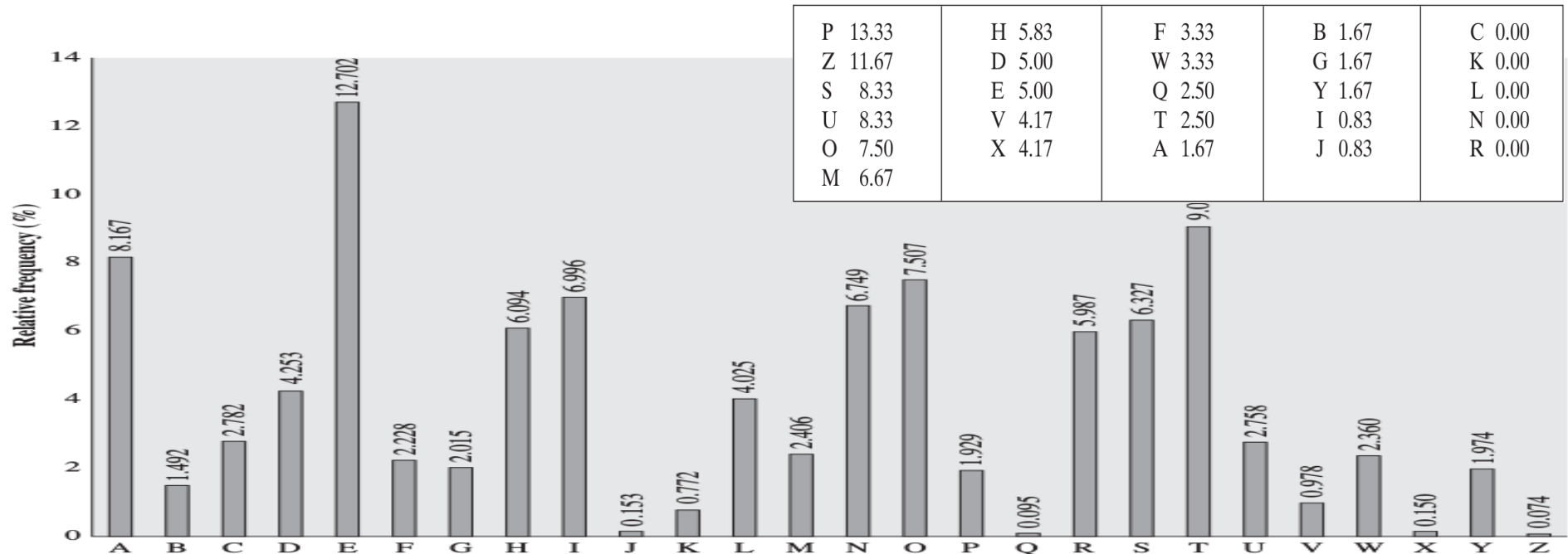
```
it was disclosed yesterday that several informal but  
direct contacts have been made with political  
representatives of the viet cong in moscow
```

Breaking Monoalphabetic Ciphers

- If the cryptanalyst knows the nature of the plaintext (e.g., noncompressed English text), then the analyst can **exploit the regularities of the language**.
- Relative frequency of the letters can be determined and compared to a **standard frequency distribution** for English.



Breaking Monoalphabetic Ciphers (contd.)



- Comparing the two distributions, it seems likely: -

- P and Z $\Leftrightarrow$ plain letters e and t
- S, U, O, M, H $\Leftrightarrow$ {a, h, i, n, o, r, s}.
- (A, B, G, Y, I, J) $\Leftrightarrow$ {b, j, k, q, v, x, z}

UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFPESXUDBMETSXAIZ
t a e e t e a t h a t e e a a
VUEPHZHMDSZSHZOWSFPAPPDTSVPQUZWYMXUZUHSX
e t t a t h a e e e a e t h t a
EPYEPOPDZSZUFPOMBZWPFUPZHMDJUDTMOHMQ
e e e t a t e t h e t

Make some **tentative assignments** and fill in the plaintext to see if it looks like a “skeleton” of a message.

- A powerful tool is to look at the frequency of two-letter combinations, called **digrams**.
- The most common digram pair is th $\Leftrightarrow$ ZW
- we can equate P with e, and thus ZWP appears to be “the”.

Substitution Techniques (contd.)

Playfair Cipher: Multiple-letter Encryption Cipher

Letters in the plaintext are treated as single units and translates these units into **ciphertext digrams**

The Playfair algorithm is based on the use of a **5 × 5 matrix of letters** constructed using a keyword – monarchy, for instance.

The matrix is filled in, from left to right and from top to bottom, with the keyword first, and then with the remaining letters in alphabetic order. “I” and “J” are counted as one letter.

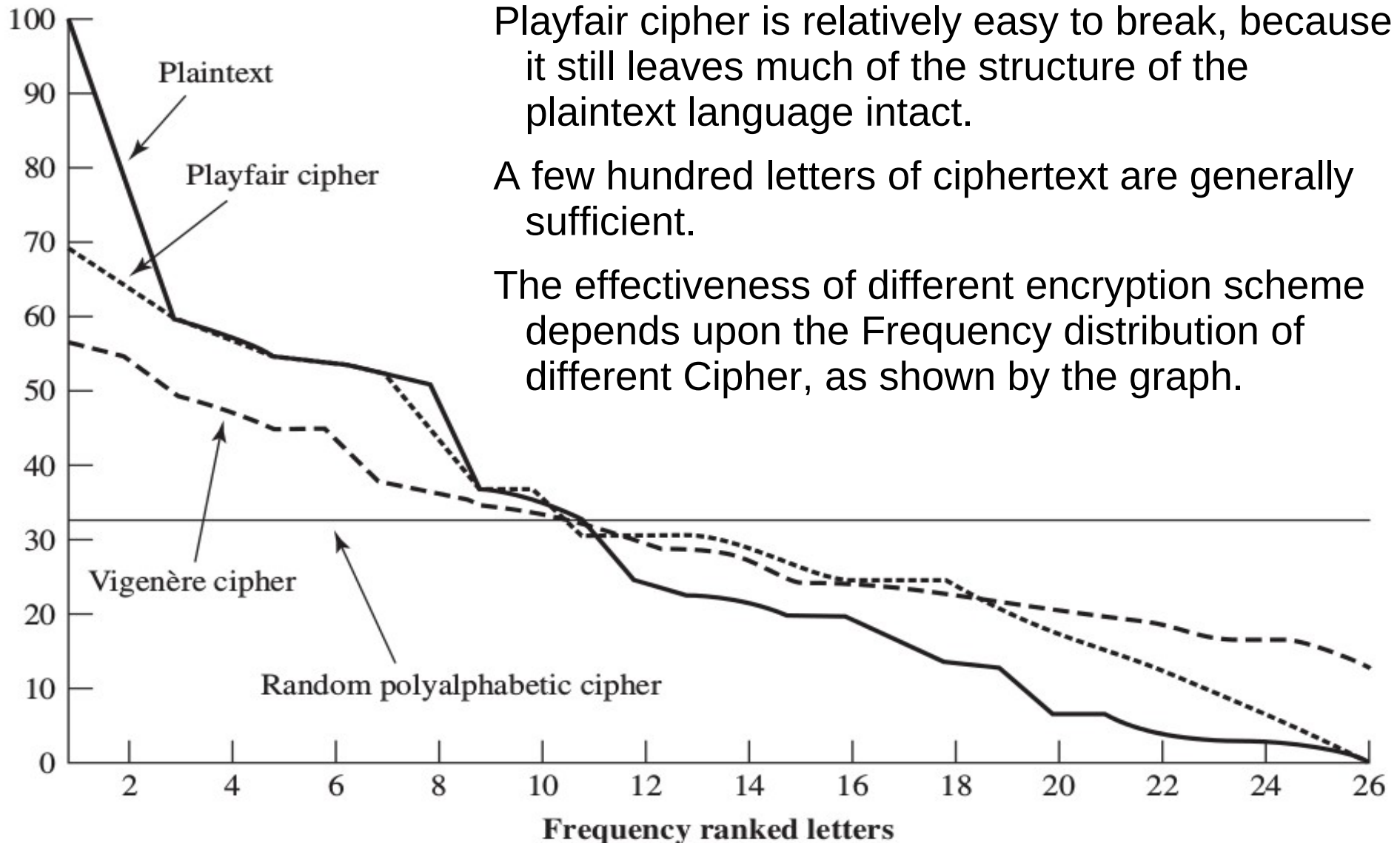
Plaintext is **encrypted two letters at a time**, according to the following rules:

1. Repeating plaintext letters that are in the same pair are separated with a filler letter, such as x, so that balloon would be treated as ba lx lo on.
2. Two plaintext letters that fall in the same row of the matrix are each replaced by the letter to the right, with the first element of the row circularly following the last. For example, ar is encrypted as RM.
3. Two plaintext letters that fall in the same column are each replaced by the letter beneath, with the top element of the column circularly following the last. For example, mu is encrypted as CM.
4. Otherwise, each plaintext letter in a pair is replaced by the letter that lies in its own row and the column occupied by the other plaintext letter. Thus, hs becomes BP and ea becomes IM (or JM, as the encipherer wishes).

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

- The Playfair cipher is a great advance over simple monoalphabetic ciphers.
- For one thing, whereas there are only 26 letters, there are $26 \times 26 = 676$ digrams
- It was used as the standard field system by the British Army in World War I and still enjoyed considerable use by the U.S. Army and other Allied forces during World War II.

Breaking Playfair Cipher



Polyalphabetic Cipher

- Uses different monoalphabetic substitutions as one proceeds through the plaintext message
- Also called polyalphabetic substitution cipher
- A set of related ***monoalphabetic substitution rules*** is used.
- A ***key determines which particular rule is chosen*** for a given transformation.

Examples: -

- Vigenère Ciphère
- One-Time Pad

Vigenère Cipher

- The best known, and one of the simplest ciphers.
- The set of related monoalphabetic substitution rules consists of the **26 Caesar ciphers with shifts of 0 through 25**.
- Each cipher is denoted by **a key letter**, which is the ciphertext letter that substitutes for the plaintext letter a. Thus, a Caesar cipher with a shift of 3 is denoted by the key value d.
- Assume a sequence of **plaintext letters** $P = p_0, p_1, p_2, \dots, p_{n-1}$ and a key consisting of the sequence of letters $K = k_0, k_1, k_2, \dots, k_{m-1}$, where typically $m < n$. The sequence of ciphertext letters $C = C_0, C_1, C_2, \dots, C_{n-1}$ is calculated as follows:

$$\begin{aligned} C &= C_0, C_1, C_2, \dots, C_{n-1} \\ &= E(K, P) = E[(k_0, k_1, k_2, \dots, k_{m-1}), (p_0, p_1, p_2, \dots, p_{n-1})] \\ &= (p_0 + k_0) \bmod 26, (p_1 + k_1) \bmod 26, \dots, (p_{m-1} + k_{m-1}) \bmod 26, (p_m + k_0) \bmod 26, (p_{m+1} + k_1) \bmod 26, \dots, (p_{2m-1} + k_{m-1}) \bmod 26, \dots \end{aligned}$$

- A general equation of the encryption process is

$$C_i = (p_i + k_{i \bmod m}) \bmod 26$$

- Similarly, decryption is given by

$$p_i = (C_i - k_{i \bmod m}) \bmod 26$$

Vigenère Cipher: Illustration

key: *deceptivedeceptivedeceptive*
 plaintext: *were discovered save yourself*
 ciphertext: *ZICVTWQNGRZGVTWAVZHCQYGLMGJ*

Expressed numerically, we have the following

key	3	4	2	4	15	19	8	21	4	3	4	2	4	15
plaintext	22	4	0	17	4	3	8	18	2	14	21	4	17	4
ciphertext	25	8	2	21	19	22	16	13	6	17	25	6	21	19

key	19	8	21	4	3	4	2	4	15	19	8	21	4
plaintext	3	18	0	21	4	24	14	20	17	18	4	11	5
ciphertext	22	0	21	25	7	2	16	24	6	11	12	6	9

- There are multiple ciphertext letters for each plaintext letter, one for each unique letter of the keyword.
- An improvement is achieved over the Playfair cipher, but considerable frequency information remains.

Breaking Vigenère Cipher

- Depends on determining the ***length of the keyword***
- If two ***identical sequences of plaintext*** letters occur at a distance that is an integer multiple of the keyword length, they will generate identical ciphertext sequences e.g.
 - » two instances of the sequence “red” are separated by nine (9) character positions
 - » in both cases, the ciphertext sequence is VTW
 - » An analyst looking at only the ciphertext would detect the repeated sequences VTW at a displacement of 9 and make the assumption that the keyword is either three or nine letters in length
 - » By looking for ***common factors in the displacements*** of the various sequences, the analyst should be able to make a good ***guess of the keyword length***.

Improved Vigenère Cipher (contd.)

- The periodic nature of the keyword can be eliminated by using a nonrepeating keyword that is as long as the message itself.
- Vigenère proposed what is referred to as an ***autokey system***, in which a keyword is concatenated with the plaintext itself to provide a running key

key:	<i>deceptivewearediscoveredsav</i>
plaintext:	<i>wearediscoveredsaveyourself</i>
ciphertext:	ZICVTWQNGKZEIIGASXSTSLVVWLA

- Even this scheme is vulnerable to cryptanalysis.
- Because the ***key and the plaintext share the same frequency distribution*** of letters, a statistical technique can be applied just as in case of other polyalphabetic ciphers.

Vernam Cipher

Introduced by an AT&T engineer named **Gilbert Vernam** in 1918.

His system **works on binary data (bits)** rather than letters. The system can be expressed as

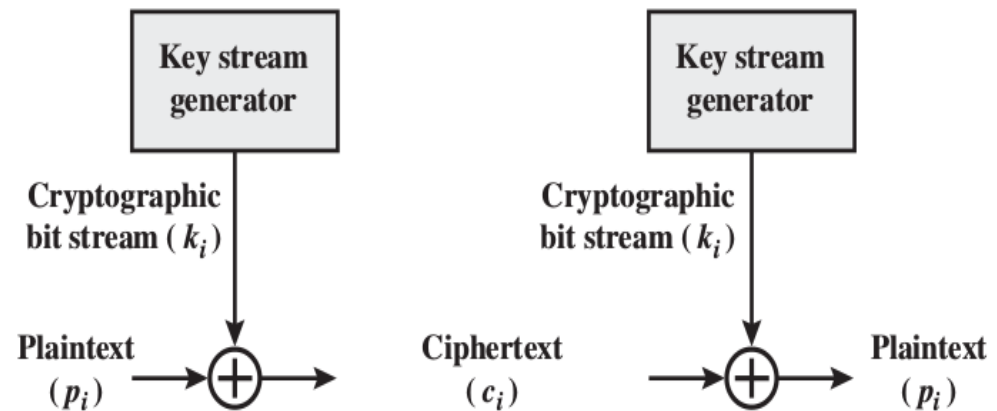
$$c_i = p_i \oplus k_i$$

where

p_i = i^{th} binary digit of plaintext

k_i = i^{th} binary digit of key

c_i = i^{th} binary digit of ciphertext



The ciphertext is generated by performing the **bitwise XOR of the plaintext and the key**.

Because of the properties of the XOR, decryption simply involves the same bitwise operation:

$$p_i = c_i \oplus k_i$$

Vernam proposed the use of **a running loop of tape** that eventually repeated the key, so that the system actually worked with a very long but repeating keyword.

Although such a scheme, with a long key, **presents formidable cryptanalytic difficulties**, it can be **broken with sufficient ciphertext**, the use of known or probable plaintext sequences, or both.

Substitution Techniques (contd.)

One-Time Pad

An Army Signal Corp officer, Joseph Mauborgne, proposed an ***improvement to the Vernam cipher.***

Mauborgne suggested using ***a random key that is as long as the message,*** so that the key need not be repeated.

In addition, the key is to be used to encrypt and decrypt a single message, and then is discarded.

Each ***new message requires a new key*** of the same length as the new message, and hence called one-time pad.

It produces random output that bears no statistical relationship to the plaintext.

Considered “***Unbreakable***” - As the ciphertext contains no information whatsoever about the plaintext, there is simply no way to break the code.

One-Time Pad : Example

Suppose that we are using a ***Vigenère scheme with 27 characters*** in which the twenty-seventh character is the space character, but with a one-time key that is as long as the message

Consider the ciphertext :

ANKYODKYUREPFJBYOJDSPLREYIUNOFDOIUERFPLUYTS

Case 1:

ciphertext: ANKYODKYUREPFJBYOJDSPLREYIUNOFDOIUERFPLUYTS

key: pxlmvmsydofuyrvzwc tnlebncvgdupahfzzlmnyih

plaintext: mr mustard with the candlestick in the hall

Case 2:

ciphertext: ANKYODKYUREPFJBYOJDSPLREYIUNOFDOIUERFPLUYTS

key: mfugpmiydgaxgoufhklmhsqdgogtewbqfgyovuhwt

plaintext: miss scarlet with the knife in the library

Question: which is the correct key? (No answer)

Is One-Time pad breakable?

The security of the one-time pad is entirely due to the ***randomness of the key***.

If the stream of characters that constitute the key is truly random, then the stream of characters that constitute the ciphertext will be truly random. Thus, there are no patterns or regularities that a cryptanalyst can use to attack the ciphertext

The one-time pad offers complete security but, in practice, has ***two fundamental difficulties***:

1. There is the practical problem of ***making large quantities of random keys***. Any heavily used system might require millions of random characters on a regular basis. Supplying truly random characters in this volume is a significant task.
2. Even more daunting is the problem of ***key distribution and protection***. For every message to be sent, a key of equal length is needed by both sender and receiver. Thus, a mammoth key distribution problem exists.

Because of these difficulties, the one-time pad is of ***limited utility*** and is useful primarily for low-bandwidth channels requiring very high security. The one-time pad is the only cryptosystem that exhibits what is referred to as ***perfect secrecy***.

Transposition Techniques: Rail Fence

A very different kind of mapping is achieved by performing some sort of ***permutation on the plaintext*** letters: called Transposition Techniques

In Rail Fence: The plaintext is written down as a ***sequence of diagonals*** and then read off as a sequence of rows

```
m e m a t r h t g p r y  
e t e f e t e o a a t
```

The encrypted message is

MEMATRHTGPRYETEFETEOAAT

- Trivial to cryptanalyze

Transposition Techniques

Rectangular Scheme

Write the **message in a rectangle**, row by row, and read the message off, column by column, but **permute the order of the columns**. The order of the columns then becomes the key to the algorithm

Key:	4	3	1	2	5	6	7																				
Plaintext:	a	t	t	a	c	k	p																				
	o	s	t	p	o	n	e																				
	d	u	n	t	i	l	t																				
	w	o	a	m	x	y	z																				
Ciphertext:	T	T	N	A	P	T	M	T	S	U	O	A	O	D	W	C	O	I	X	K	N	L	Y	P	E	T	Z

A pure transposition cipher is easily recognized because it has the **same letter frequencies** as the original plaintext.

Cryptanalysis is fairly straightforward and involves **laying out the ciphertext in a matrix** and **playing around with column positions**.

Digram and trigram **frequency tables** may be useful.

Transposition Techniques

More Secure Scheme

Perform more than one stage of transposition
The result is a more complex permutation that is not easily reconstructed.

Key:	4	3	1	2	5	6	7
Input:	t	t	n	a	a	p	t
	m	t	s	u	o	a	o
	d	w	c	o	i	x	k
	n	l	y	p	e	t	z
Output:	NSCYAUOPTTWLTMDNAOIEPAXTTOKZ						

the original sequence of letters is

01 02 03 04 05 06 07 08 09 10 11 12 13 14
15 16 17 18 19 20 21 22 23 24 25 26 27 28

After the first transposition, we have

03 10 17 24 04 11 18 25 02 09 16 23 01 08
15 22 05 12 19 26 06 13 20 27 07 14 21 28

But after the second transposition, we have

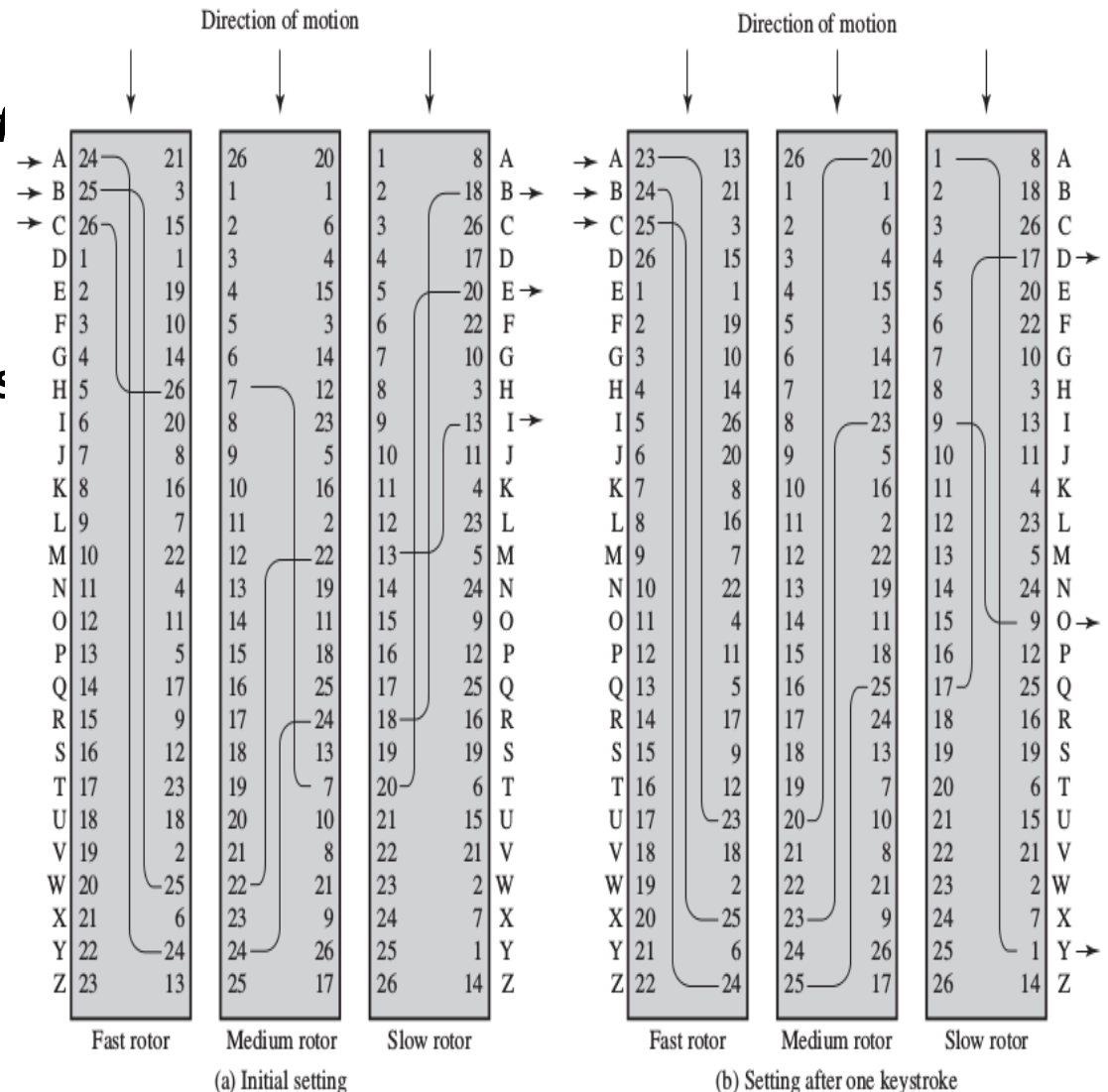
17 09 05 27 24 16 12 07 10 02 22 20 03 25
15 13 04 23 19 14 11 01 26 21 18 08 06 28

Rotor Machines

The most important application of the principle of **multiple stages of encryption** was a class of systems known as rotor machines

The machine consists of **a set of independently rotating cylinders** through which electrical pulses can flow. Each cylinder has **26 input pins and 26 output pins**, with internal wiring that connects each input pin to a unique output pin.

If we **associate each input and output pin with a letter of the alphabet**, then a single cylinder defines a **monoalphabetic substitution**.

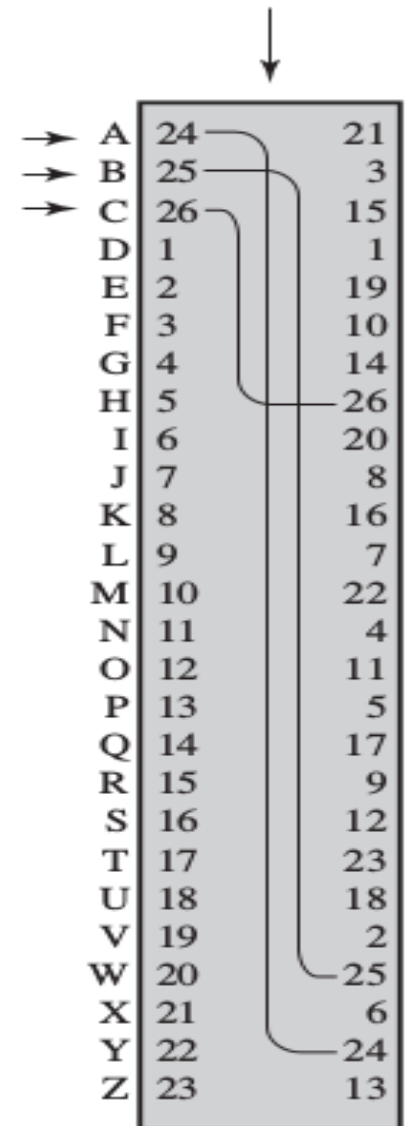


Rotor Machines: Single Cylinder

Consider a machine with a single cylinder. After ***each input key*** is depressed, the ***cylinder rotates one position***, so that the internal connections are shifted accordingly.

Thus, a different monoalphabetic substitution cipher is defined. After 26 letters of plaintext, the cylinder would be back to the initial position. Thus, we have a ***polyalphabetic substitution algorithm with a period of 26***.

A single-cylinder system is trivial and does not present a formidable cryptanalytic task.



Rotor Machines: Multiple Cylinders

The **output pins of one cylinder are connected to the input pins of the next.**

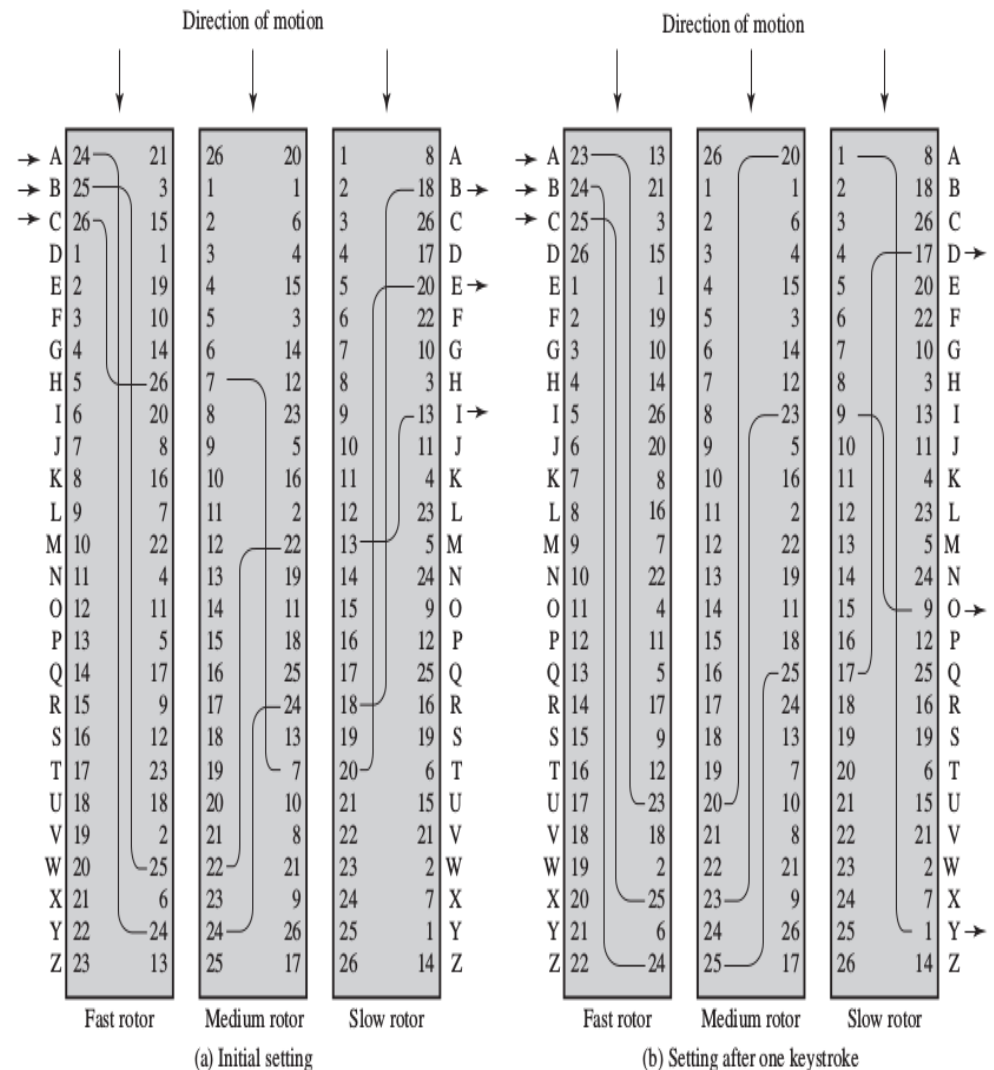
The **left half of the figure** shows a position in which the **input from the operator to the first pin** (plaintext letter a) is routed through the three cylinders to appear at the **output of the second pin** (ciphertext letter B)

With multiple cylinders, the one closest to the operator input rotates one pin position with each keystroke.

The **right half of the figure** shows the **system's configuration after a single keystroke**. For every complete rotation of the inner cylinder, the middle cylinder rotates one pin position. Finally, for every complete rotation of the middle cylinder, the outer cylinder rotates one pin position.

There are $26 * 26 * 26 = 17,576$ **different substitution** alphabets used before the system repeats. The addition of fourth and fifth rotors results in periods of 456,976 and 11,881,376 letters, respectively.

The significance of the rotor machine today is that it points the way to the most widely used cipher ever: the **Data Encryption Standard (DES)**.



Block Ciphers

A block cipher is one in which *a block of plaintext is treated as a whole* and used to produce *a ciphertext block of equal length*.

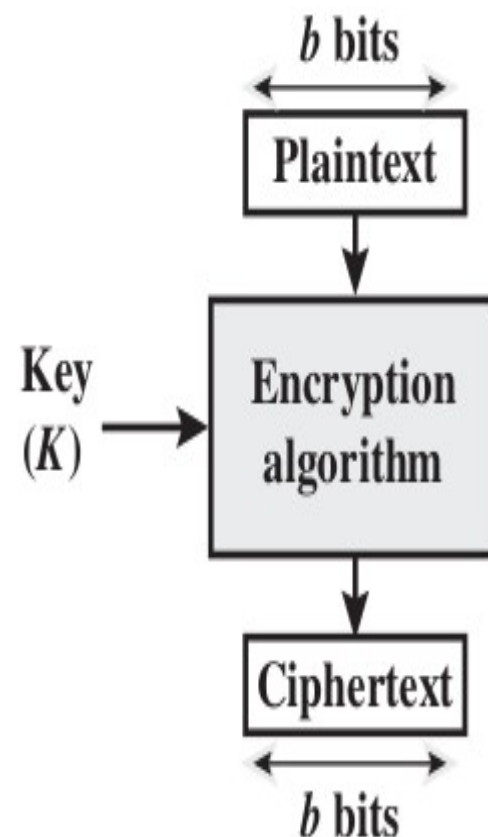
Many block ciphers *have a Feistel structure*. Such a structure consists of a number of *identical rounds of processing*.

Typically, a *block size of 64 or 128 bits* is used. As with a stream cipher, the two users *share a symmetric encryption key*.

A block cipher operates on a plaintext block of n bits to produce a ciphertext block of n bits.

There are 2^n possible different plaintext blocks.

Using some of the modes of operation (e.g. Cipher feedback mode or Output feedback mode), a block cipher can also be used to *achieve the same effect as a stream cipher*.



Ideal Block Cipher: Problems

With a small block size, e.g. $n = 4$, the system becomes ***equivalent to a classical substitution cipher***. i.e. no more secure statistically.

An arbitrary reversible substitution cipher (the ideal block cipher) for a large block size is ***not practical from an implementation and performance point of view***.

When $n=4$ block of plaintext, the straightforward key size is (4 bits * 16 rows =) 64 bits long, i.e. $n \cdot 2^n$. So, for a 64-bit block, key size will be $64 \cdot 2^{64} = 2^{70} = 10^{21}$ approx.

Motivation for Feistel cipher: -

What is needed is an ***approximation to the ideal block cipher system for large n*** , built up out of components that are ***easily realizable***.

Plaintext	Ciphertext
0000	1110
0001	0100
0010	1101
0011	0001
0100	0010
0101	1111
0110	1011
0111	1000
1000	0011
1001	1010
1010	0110
1011	1100
1100	0101
1101	1001
1110	0000
1111	0111

Feistel Cipher Structure

Horst Feistel proposed the concept of a *product cipher*;

- *the execution of two or more simple ciphers in sequence*
- *final product is cryptographically stronger than any of the component ciphers.*

Feistel cipher structure consists of a *block cipher* with a *key length of k bits* and *a block length of n bits*;

- Has to undergo 2^k *possible transformations* only (2^n transformations in ideal block cipher).

Feistel cipher makes use of *substitutions and permutations* alternatively : -

Substitution: Each plaintext element or group of elements is uniquely *replaced by a corresponding ciphertext* element or group of elements.

Permutation: A sequence of plaintext elements is *replaced by a permutation of that sequence*. (i.e. the order of the elements is changed.)

The Feistel cipher structure, which is based on Shannon's proposal of *Diffusion and Confusion* in 1945, is the structure used by many significant symmetric block ciphers currently in use.

Feistel Cipher (contd.)

Inputs to the encryption algorithm are a **plaintext block of length $2w$ bits** and a **key K** .

The plaintext block is **divided into two halves**, L_0 and R_0 .

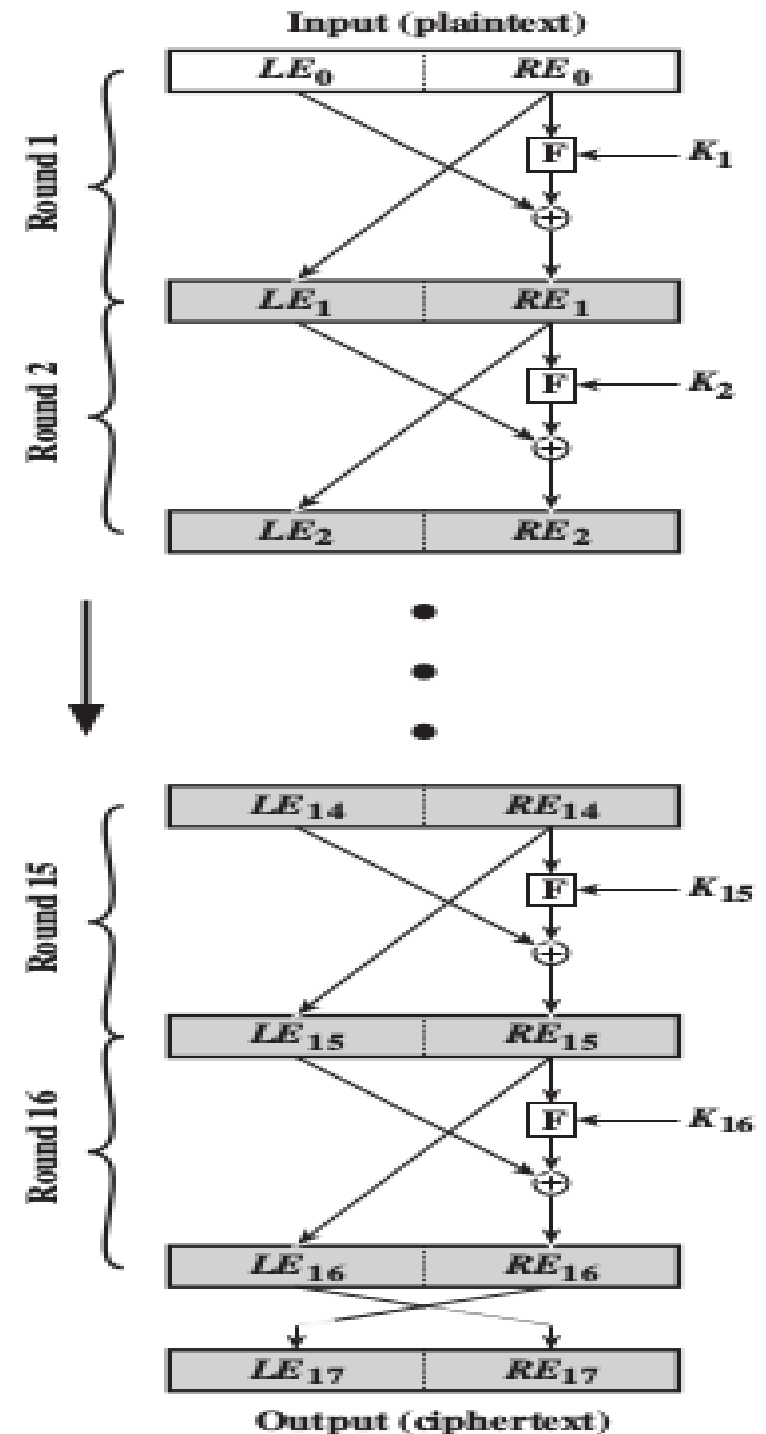
The two halves of the data **pass through n rounds** of processing and then **combine to produce the ciphertext block**.

Each round i has as inputs L_{i-1} and R_{i-1} derived from the previous round, as well as a subkey K_i derived from the overall K .

A **substitution is performed on the left half of the data**.

- Apply a **round function F** to the right half of the data and then take the X-OR of the output of that function and the left half of the data.
- F has the same general structure for each round but is parameterized by the round subkey K_i .

Following this substitution, a **permutation is performed** that consists of the interchange of the two halves of the data.

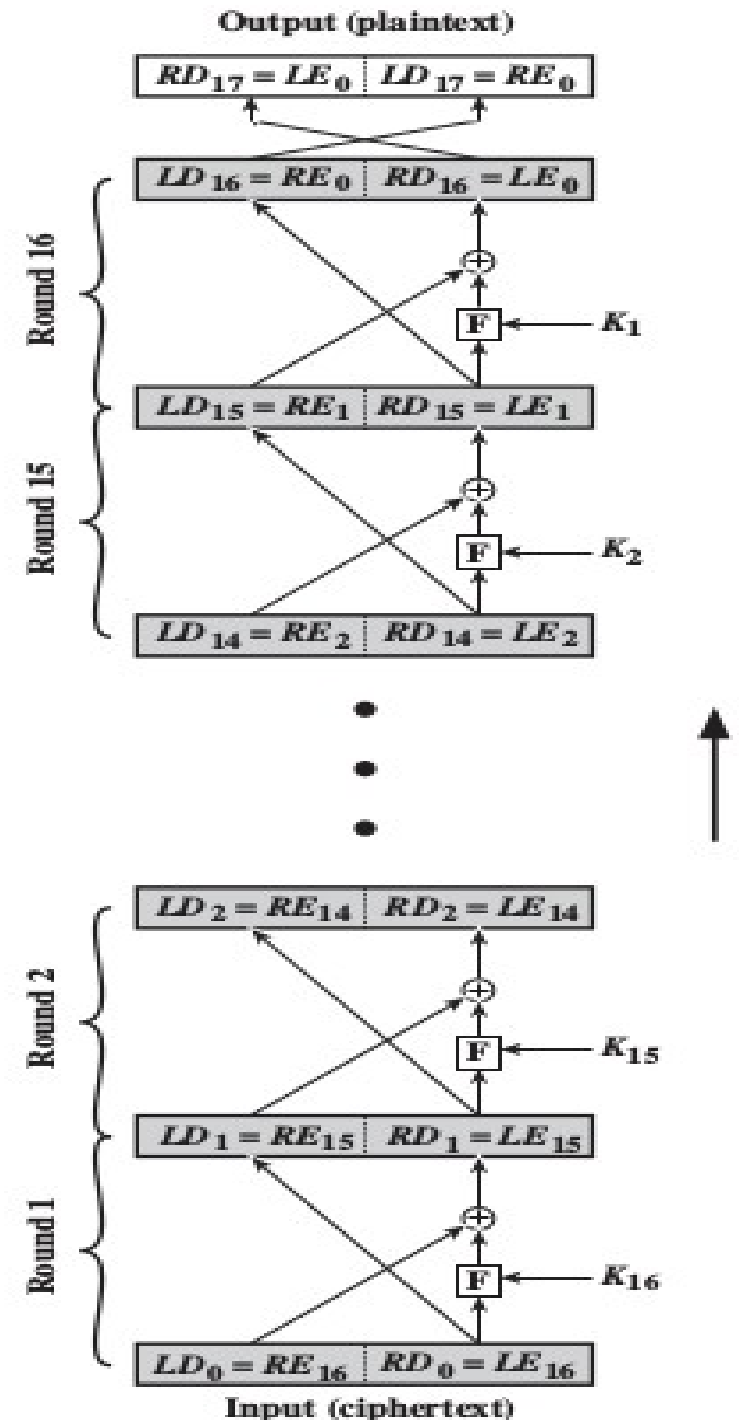


Feistel Cipher (contd.)

Decryption has *same process as the encryption*; only the input keys parameterize it.

The *subkeys are used in reverse order*, i.e. K_{16} is used in the first round of decryption.

The *first round of decryption reverses the last round of encryption*, the second round of decryption reverses the second last round of encryption, and so on.



Feistel Cipher: Analysis

The *realization of a Feistel network* depends on : -

- Block size:
 - Larger block sizes mean greater security but reduced speed for algorithm.
- Key size:
 - Larger key size means greater security but decreases encryption/ decryption speed.
- Number of rounds:
 - Multiple rounds offer increasing security. A typical size is 16 rounds.
- Subkey generation algorithm:
 - Greater complexity leads to greater difficulty of cryptanalysis.
- Round function F:
 - A complex function means greater resistance to cryptanalysis.

Other considerations : -

- Fast software encryption/decryption:
 - In case of a hardware implementation, the speed of execution of the algorithm becomes a concern.
- Easy analysis of algorithm:
 - If the algorithm can be concisely and clearly explained, it is easier to analyze that algorithm for cryptanalytic vulnerabilities.

The Data Encryption Standard (DES)

First developed in 1974 by IBM and NSA (based on LUCIFER that was 64-bit encryption with 128 bit key size).

Also known as ***Data Encryption Algorithm*** (DEA) that was adopted in 1977 by National Bureau of Standards (presently called NIST) as the Data Encryption Standard.

Data are encrypted in ***64-bit blocks*** using a ***56-bit key***.

The same steps, with the same key, are used to reverse the encryption.

DES has the ***same structure as Feistel cipher*** except the F function and additional initial and final permutations IP and IP^{-1} , respectively.

DES Encryption

Encryption phases: -

1. Initial permutation (IP):

the 64-bit plaintext passes through and rearranges the bits to produce the permuted input

2. Sixteen rounds of substitutions and permutations.

The output of the last round consists of 64 bits that is a function of the input plaintext and the key.

The left and right halves of the output are swapped to produce the preoutput.

3. Final permutation (IP^{-1})

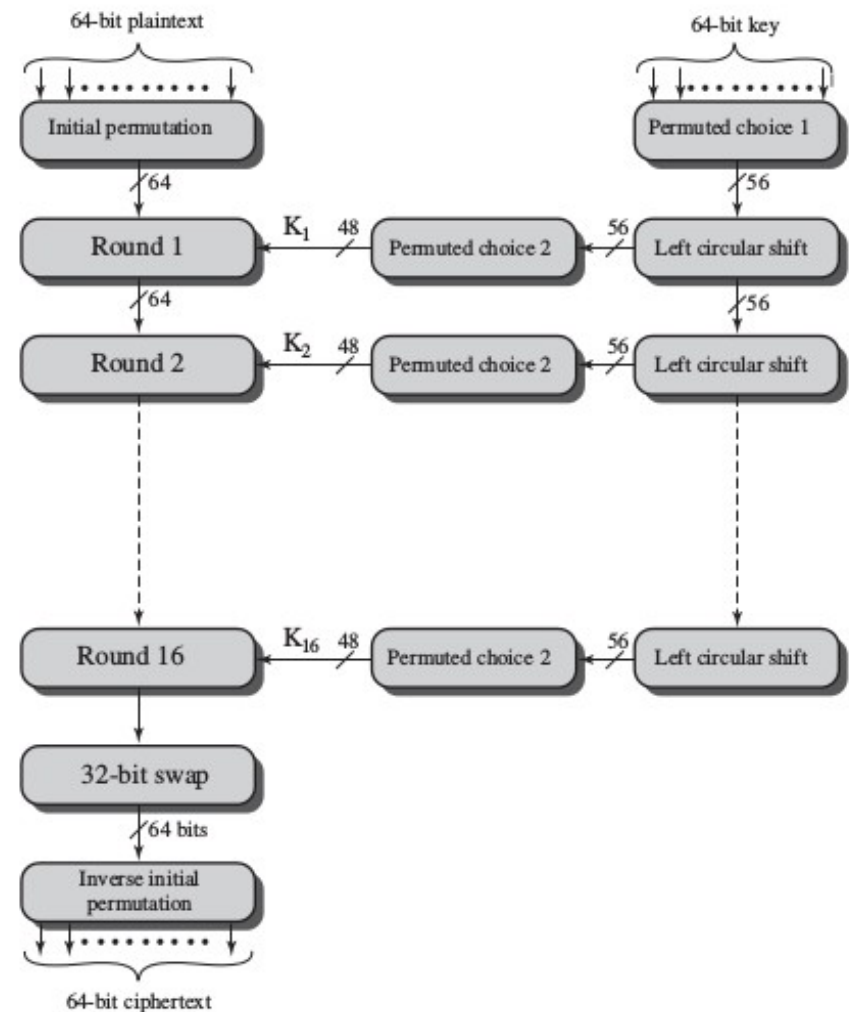
Inverse of the initial permutation function.

Key scheduling: -

Initially, the 64-bit key is passed through a permutation function.

Then, for each of the sixteen rounds, a subkey (K_i) is produced by the combination of a left circular shift and a permutation

The permutation function is the same for each round, but a different subkey is produced because of the repeated shifts of the key bits.



DES : IP and IP⁻¹

Initial and final permutations are defined by tables.

Both tables consist of 64 bits each numbered from 1 to 64.

Each entry in the permutation table indicates the position of a numbered input bit in the output.

(a) Initial Permutation (IP)

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

(b) Inverse Initial Permutation (IP⁻¹)

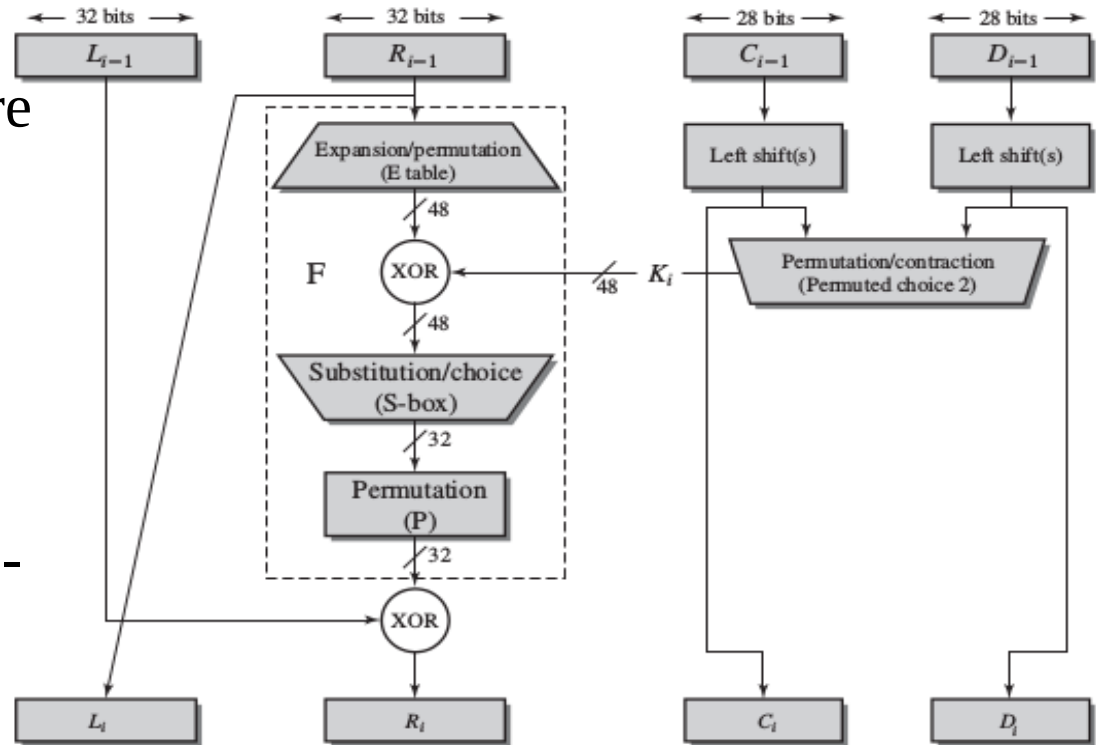
40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

DES : Internals of a Single Round

Input R is expanded to 48 bits
(using Exp^n Permut^n table), where
16 of the R bits are duplicated.

Expanded 48 bits are X-ORed with
 K_i .

X-ORed 48-bits pass through a
substitution function (by using S-
box) that produces a 32-bit
output.



Those 32-bits are permuted as
defined by the permut^n table.

(d) Permutation Function (P)

16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

(c) Expansion Permutation (E)

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

DES : S-box Substitution

- The substitution consists of a set of eight S-boxes, each of which accepts 6 bits as input and produces 4 bits as output.
- The first and last bits of the input to box S_i form a 2-bit binary number to select one of four substitutions defined by the four rows in the table for S_i .
- The middle four bits select one of the sixteen columns

S_1

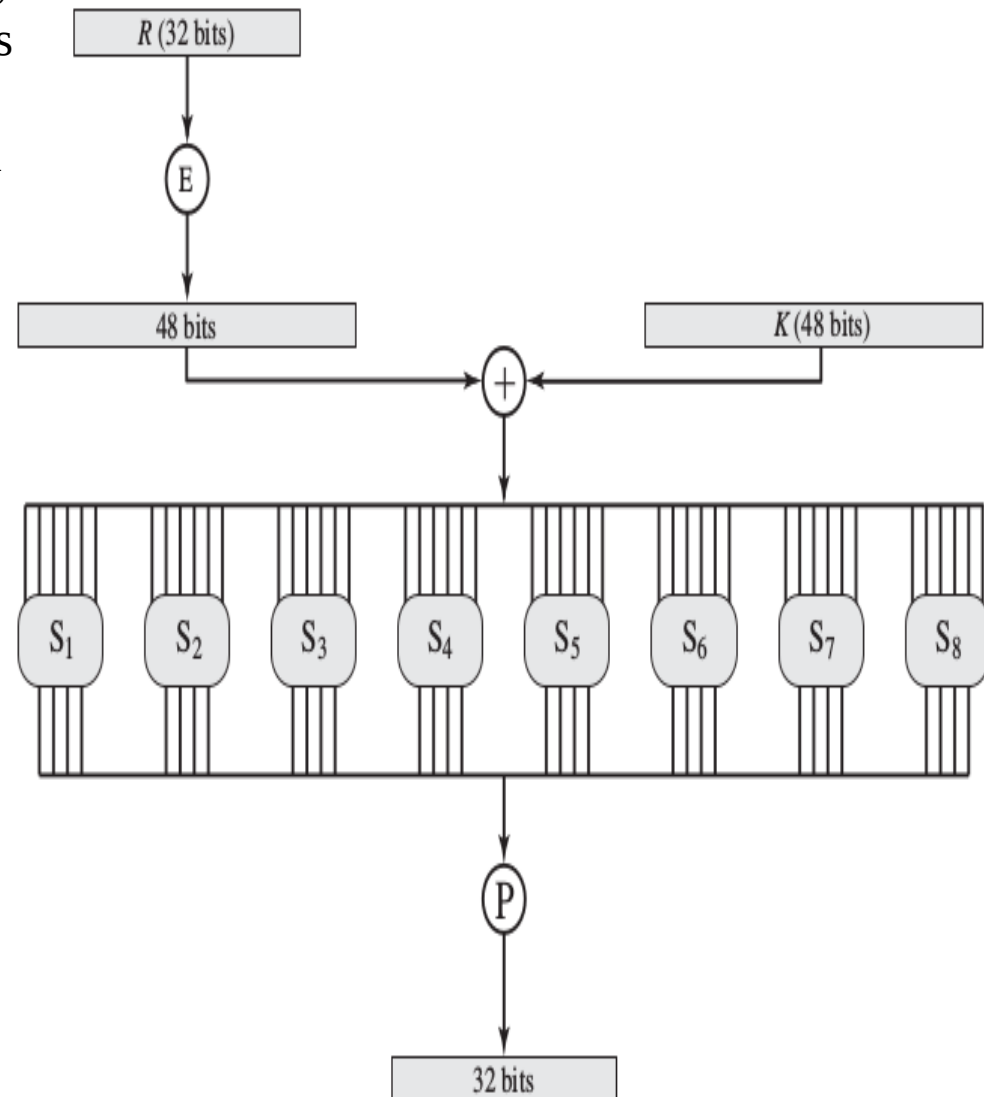
14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

...

...

S_8

13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11



DES : Key Generation

A 64-bit key(numbered 1 – 64) is used as input to the algorithm.

Every eighth bit (8, 16,... ... , 56, 64) is ignored.

The key is first subjected to a permutation governed by the table Permuted Choice One.

The resulting 56-bit key is then treated as two 28-bit quantities, labeled C_0 and D_0 .

At each round, C_{i-1} and D_{i-1} are separately subjected to a circular left shift or (rotation) of 1 or 2 bits, as governed by Shift tables.

These shifted values serve as input to the table Permuted Choice Two, which produces a 48-bit output that serves as input to the function F.

They also serve as input to the next round.

(b) Permuted Choice One (PC-1)

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

(c) Permuted Choice Two (PC-2)

14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

(d) Schedule of Left Shifts

Round Number	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bits Rotated	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

DES : Security Issues

1. Key-length

There are 2^{56} possible keys ($=7.2 * 10^{16}$ keys approx) - a brute-force attack is impractical.

In July 1998, a special-purpose machine “DES cracker” that was made by Electronic Frontier Foundation (EFF) with \$250,000 broke a DES encryption in less than three days.

2. S-boxes

The design criteria for S-boxes, and for the entire algorithm, were not made public, So, there is a suspicion that the boxes were constructed in such a way that cryptanalysis is possible for an opponent who knows the weaknesses in the S-boxes.

Other successful attacks on DES: -

Differential analysis : required 2^{47} (x,y) pairs

Linear analysis : required 2^{43} (x,y) pairs

Triple DES

Multiple encryption techniques with multiple keys – 2DES and 3DES.

1. Double DES : encrypts twice with two 58-bit keys.

$$C = E(K_2, E(K_1, P))$$

key length is $56 * 2 = 112$ bits

2. Triple DES with double keys : encrypts thrice with two 58-bit keys.

$$C = E(K_1, D(K_2, E(K_1, P)))$$

key length still is $56 * 2 = 112$ bits

3. Triple DES with triple keys : encrypts thrice with three 58-bit keys.

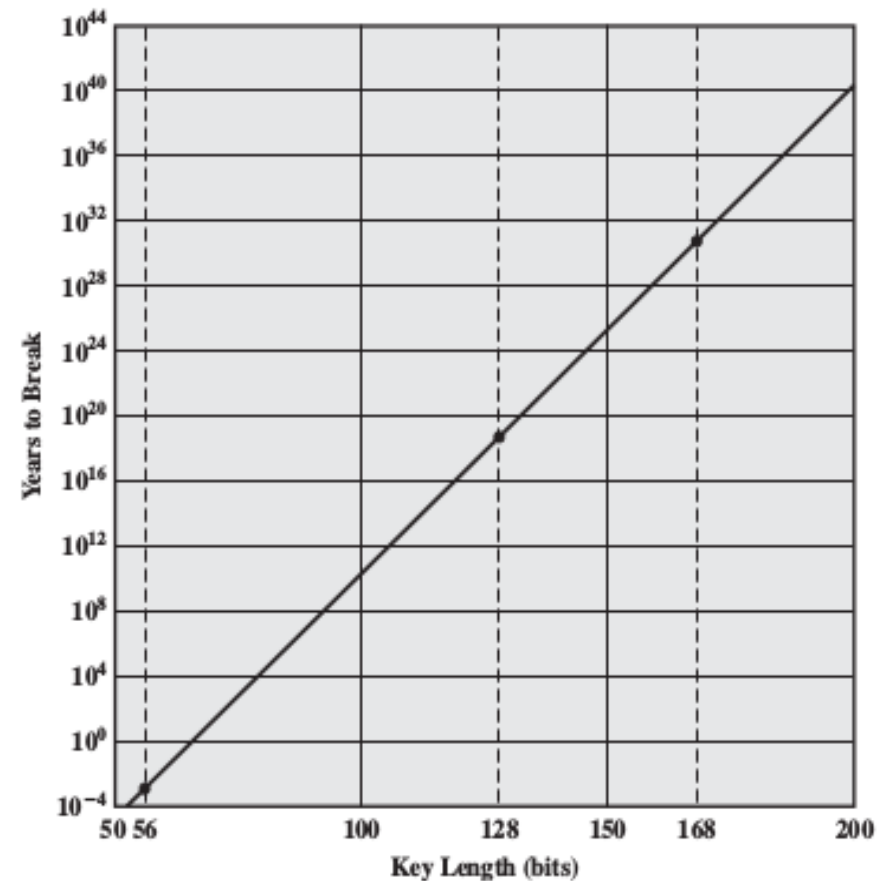
$$C = E(K_3, D(K_2, E(K_1, P)))$$

Backward compatibility with DES is provided by putting

$$K_3 = K_2 \text{ or } K_1 = K_2$$

key length now becomes $56 * 3 = 168$ bits

Resistance against Brute-force attack



Advanced Encryption Standard (AES)

AES is the most widely used symmetric cipher today.

Internet security standard IPsec, TLS, the Wi-Fi encryption standard IEEE 802.11i, the secure shell network protocol SSH (Secure Shell), the Internet phone Skype etc. use AES.

The AES selection procedure: -

On Sep. 12, 1997, NIST called for proposals for a new Advanced Encryption Standard (AES). Fifteen candidate algorithms were submitted by researchers from several countries by August 20, 1998.

On August 9, 1999, five finalist algorithms were selected: -

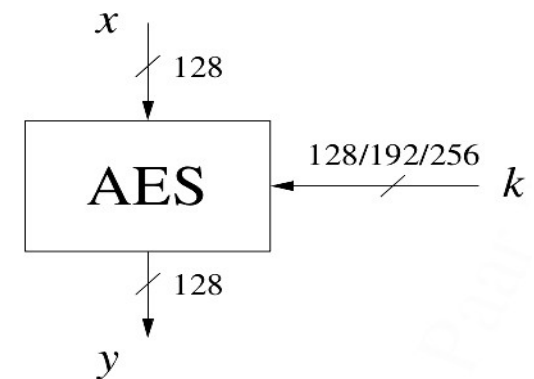
- Mars by IBM Corporation
- RC6 by RSA Laboratories
- Rijndael, by Joan Daemen and Vincent Rijmen
- Serpent, by Ross Anderson, Eli Biham and Lars Knudsen
- Twofish, by Bruce Schneier, John Kelsey, Doug Whiting, David Wagner, ChrisHall and Niels Ferguson

On October 2, 2000, NIST announced that it had chosen Rijndael as the AES.

On November 26, 2001, AES was formally approved as a US federal standard.

In 2003, the US National Security Agency (NSA) announced that it allows AES to encrypt classified documents up to the level SECRET for all key lengths, and up to the TOP SECRET level for key lengths of either 192 or 256 bits.

(Before, only non-public algorithms had been used for the encryption of classified document.)



key lengths	# rounds = n_r
128 bit	10
192 bit	12
256 bit	14

AES Overview

Unlike Feistel cipher, AES encrypts all 128 bits in one iteration.

AES consists of three different types of layers, each of them manipulates all 128 bits of the data path (also called states).

(Each round, with the exception of the first, consists of all three layers.)

Key Addition layer

A 128-bit round key, or subkey, which has been derived from the main key in the key schedule, is XORed to the state (or data path).

Byte Substitution layer (S-Box)

Each element of the state is nonlinearly transformed (by providing confusion) using lookup tables with **special mathematical properties**.

Diffusion layer

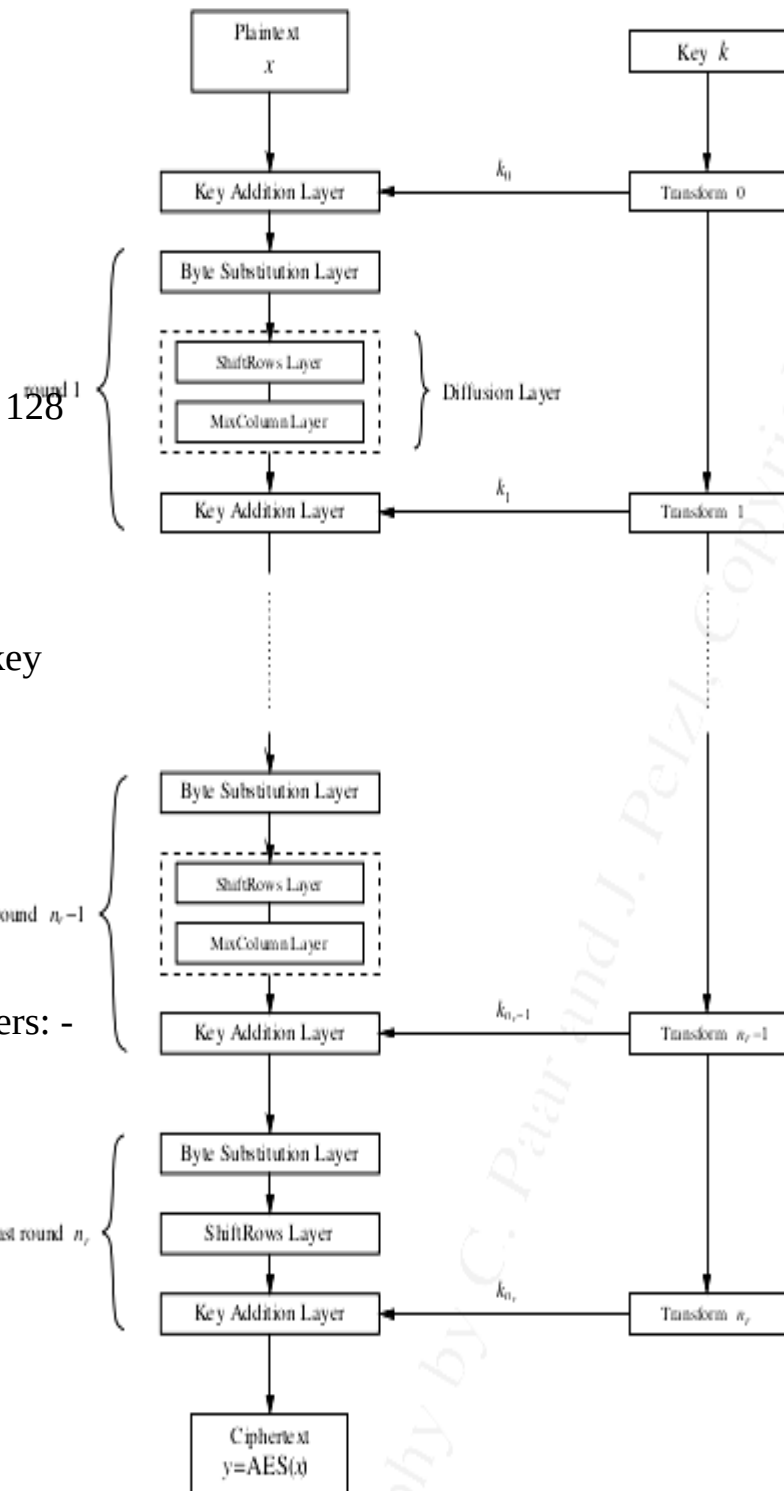
It provides **diffusion** over all state bits. It further consists of two sub-layers: -

The **ShiftRows layer** permutes the data on a byte level.

The **MixColumn layer** combines blocks of four bytes.

Key Scheduling

Similar to DES, the key schedule computes round keys, or subkeys, $(k_0, k_1, \dots, k_{nr})$ from the original key.



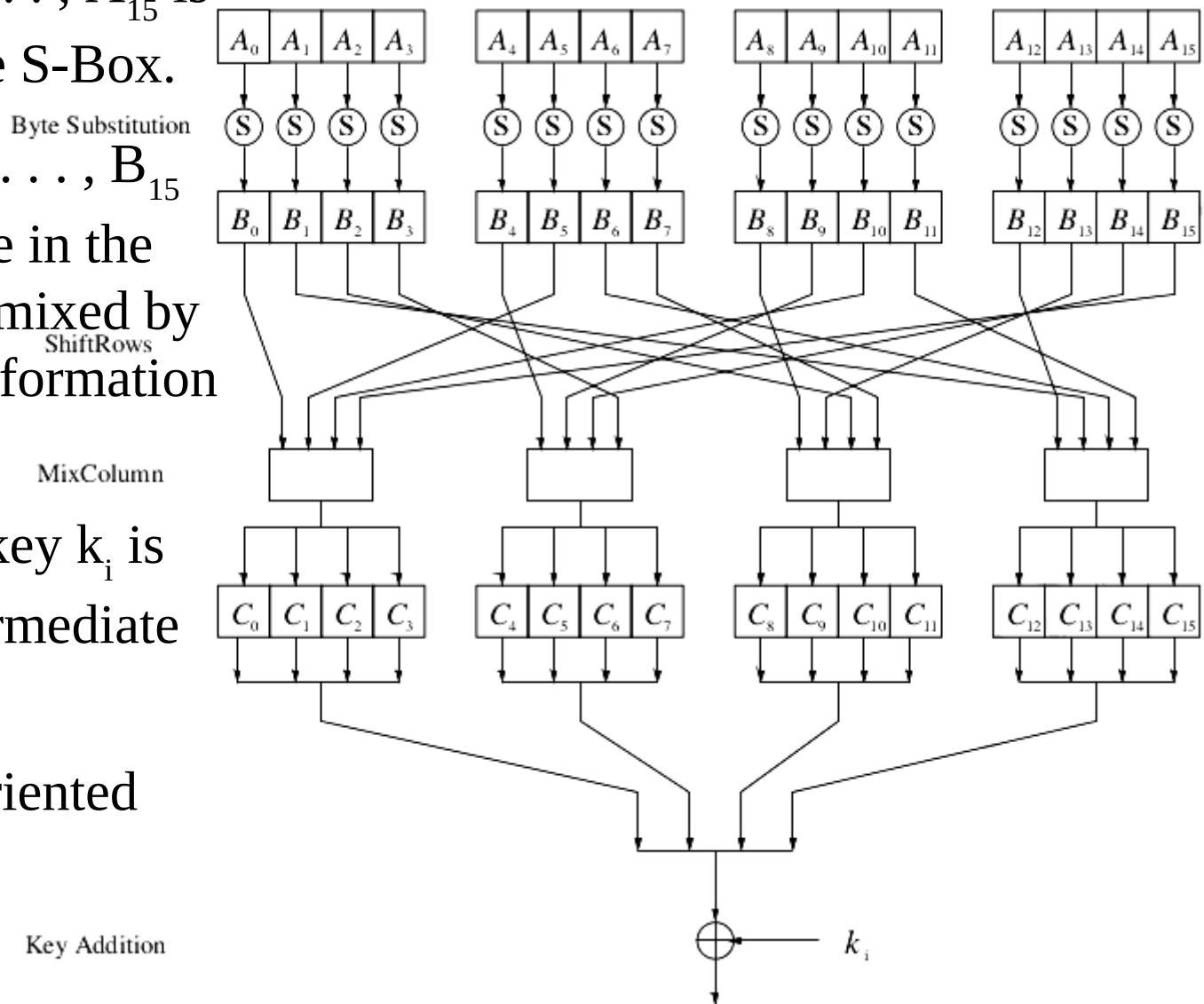
AES Internal Structure

The 16-byte input $A_0, \dots, A_{15}$ is fed byte-wise into the S-Box.

The 16-byte output $B_0, \dots, B_{15}$ is permuted byte-wise in the ShiftRows layer and mixed by the MixColumn transformation $c(x)$.

Finally, the 128-bit subkey k_i is XORed with the intermediate result.

(Note : AES is a byte-oriented cipher.)



Byte Substitution Layer

The Byte Substitution layer is viewed as a row of 16 parallel S-Boxes, each with 8 input and output bits.

(Note: all 16 S-Boxes are identical, unlike DES.)

Each state byte A_i is substituted by another byte B_i i.e.

$$S(A_i) = B_i.$$

The S-Box substitution is a bijective mapping, i.e., each of the $2^8 = 256$ possible input elements is one-to-one mapped to one output element, by allowing to uniquely reverse the S-Box for decryption.

S-Box is usually realized as a 256-by-8 bit lookup table with fixed entries.e.g.

$$S((C2)_{\text{hex}}) = (25)_{\text{hex}}$$

$$S(1100\ 0010) = (0010\ 0101)$$

$GF(2^8)$ provides a high degree of nonlinearity and affine mapping “destroys” the algebraic structure of the Galois field.

	y															
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

{Mathematical description}



{Both steps for all 256 possible input elements of the S-Box are computed and stored to obtain the table.}

Substitution Example: $S(1100\ 0010) = (0010\ 0101)$

1. Galois field inversion ($B_i' = A^{-1}$)

Here, both A^{-1} and B_i' are considered elements in the field $GF(2^8)$ with the fixed irreducible polynomial $P(x) = x^8 + x^4 + x^3 + x + 1$.

For a given finite field $GF(2^m)$ and the corresponding irreducible reduction polynomial $P(x)$, the inverse A^{-1} of a nonzero element $A \in GF(2^m)$ is defined as:

$$A^{-1}(x) \cdot A(x) = 1 \bmod P(x).$$

We have,

$$A = (1100\ 0010) \text{ i.e. } (C2)_{\text{hex}}$$

A^{-1} is given by the table as $(2F)_{\text{hex}}$ which is $(0010\ 1111)$ in binary.

This can be verified by multiplication:

$$(x^7 + x^6 + x) \cdot (x^5 + x^3 + x^2 + x + 1) \equiv 1 \bmod P(x)$$

Multiplicative inverse table in $GF(2^8)$ for bytes xy used within the AES S-Box

	Y															
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	00	01	8D	F6	CB	52	7B	D1	E8	4F	29	C0	B0	E1	E5	C7
1	74	B4	AA	4B	99	2B	60	5F	58	3F	FD	CC	FF	40	EE	B2
2	3A	6E	5A	F1	55	4D	A8	C9	C1	0A	98	15	30	44	A2	C2
3	2C	45	92	6C	F3	39	66	42	F2	35	20	6F	77	BB	59	19
4	1D	FE	37	67	2D	31	F5	69	A7	64	AB	13	54	25	E9	09
5	ED	5C	05	CA	4C	24	87	BF	18	3E	22	F0	51	EC	61	17
6	16	5E	AF	D3	49	A6	36	43	F4	47	91	DF	33	93	21	3B
7	79	B7	97	85	10	B5	BA	3C	B6	70	D0	06	A1	FA	81	82
8	83	7E	7F	80	96	73	BE	56	9B	9E	95	D9	F7	02	B9	A4
9	DE	6A	32	6D	D8	8A	84	72	2A	14	9F	88	F9	DC	89	9A
A	FB	7C	2E	C3	8F	B8	65	48	26	C8	12	4A	CE	E7	D2	62
B	0C	E0	1F	EF	11	75	78	71	A5	8E	76	3D	BD	BC	86	57
C	0B	28	2F	A3	DA	D4	E4	0F	A9	27	53	04	1B	FC	AC	E6
D	7A	07	AE	63	C5	DB	E2	EA	94	8B	C4	D5	9D	F8	90	6B
E	B1	0D	D6	EB	C6	0E	CF	AD	08	4E	D7	E3	5D	50	1E	B3
F	5B	23	38	34	68	46	03	8C	DD	9C	7D	A0	CD	1A	41	1C

The table contains all inverses in $GF(2^8)$ modulo $P(x) = x^8 + x^4 + x^3 + x + 1$ in hexadecimal notation.

Substitution Example: $S(1100\ 0010) = (0010\ 0101)$ (contd.)

2. Affine mapping

Each byte B_i' is multiplied by a constant bit-matrix followed by the addition of a constant 8-bit vector.

Here,

$$B_i' = (b_7' b_6' b_5' b_4' b_3' b_2' b_1' b_0') \\ = (0010\ 1111).$$

So,

$$B_i = (b_7 b_6 b_5 b_4 b_3 b_2 b_1 b_0) \\ = (0010\ 0101).$$

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{pmatrix} \equiv \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} b_0' \\ b_1' \\ b_2' \\ b_3' \\ b_4' \\ b_5' \\ b_6' \\ b_7' \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{pmatrix} \pmod{2}$$

Diffusion Layer

ShiftRows Sublayer

The ShiftRows transformation cyclically shifts the rows of the state matrix by certain no. of bytes to the right, except the first row.

B_0	B_4	B_8	B_{12}
B_1	B_5	B_9	B_{13}
B_2	B_6	B_{10}	B_{14}
B_3	B_7	B_{11}	B_{15}

MixColumn Sublayer

each 4-byte column is considered as a vector and multiplied by a fixed 4×4 matrix. The matrix contains constant entries. Multiplication and addition of the coefficients is done in GF(2⁸).

All constants entries are in hex notation.

Since every input byte influences four output bytes, the MixColumn operation is the major diffusion element in AES.

The combination of the ShiftRows and MixColumn layer makes it possible that after only three rounds every byte of the state matrix depends on all 16 plaintext bytes.

B_0	B_4	B_8	B_{12}	no shift
B_5	B_9	B_{13}	B_1	← one position left shift
B_{10}	B_{14}	B_2	B_6	← two positions left shift
B_{15}	B_3	B_7	B_{11}	← three positions left shift

$$\begin{pmatrix} C_0 \\ C_1 \\ C_2 \\ C_3 \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \begin{pmatrix} B_0 \\ B_5 \\ B_{10} \\ B_{15} \end{pmatrix}$$

Example showing how the first four output bytes are computed.

MixColumn Example

Suppose, the input state to the MixColumn layer is $B = (25, 25, \dots, 25)$

In this case, only two multiplications in $GF(2^8)$ have to be done viz. $02 \cdot 25$ and $03 \cdot 25$, which can be computed in polynomial notation:

$$02 \cdot 25 = x \cdot (x^5 + x^2 + 1)$$

$$= x^6 + x^3 + x$$

$$03 \cdot 25 = (x + 1) \cdot (x^5 + x^2 + 1)$$

$$= (x^6 + x^3 + x) + (x^5 + x^2 + 1)$$

$$= x^6 + x^5 + x^3 + x^2 + x + 1$$

Since both intermediate values have a degree smaller than 8, no modular reduction with $P(x)$ is necessary.

The output bytes of C result from the following addition in $GF(2^8)$:

$$\begin{array}{rcll} 01 \cdot 25 & = & x^5 + & x^2 + 1 \\ 01 \cdot 25 & = & x^5 + & x^2 + 1 \\ 02 \cdot 25 & = & x^6 + & x^3 + x \\ 03 \cdot 25 & = & x^6 + x^5 + & x^3 + x^2 + x + 1 \\ \hline C_i & = & x^5 + & x^2 + 1, \end{array}$$

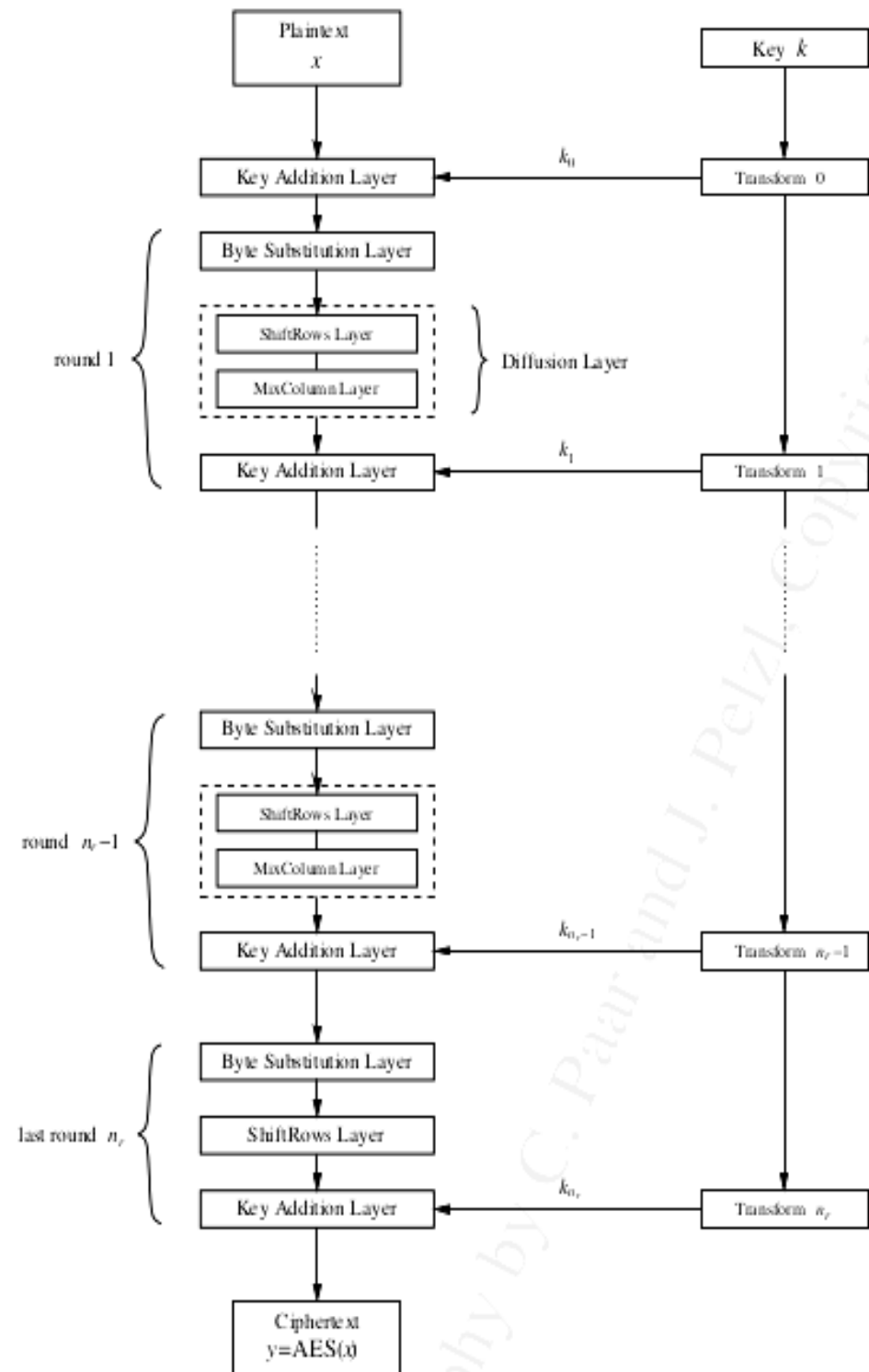
where $i = 0, \dots, 15$. This leads to the output state $C = (25, 25, \dots, 25)$

Key Addition Layer

The two inputs to the Key Addition layer are the current 16-byte state matrix and a subkey which also consists of 16 bytes (128 bits).

The two inputs are combined through a bitwise XOR operation.

Note that the XOR operation is equal to addition in the Galois field $GF(2)$.



Key Scheduling

The key schedule takes the original input key (of length 128, 192 or 256 bits) and derives the subkeys.

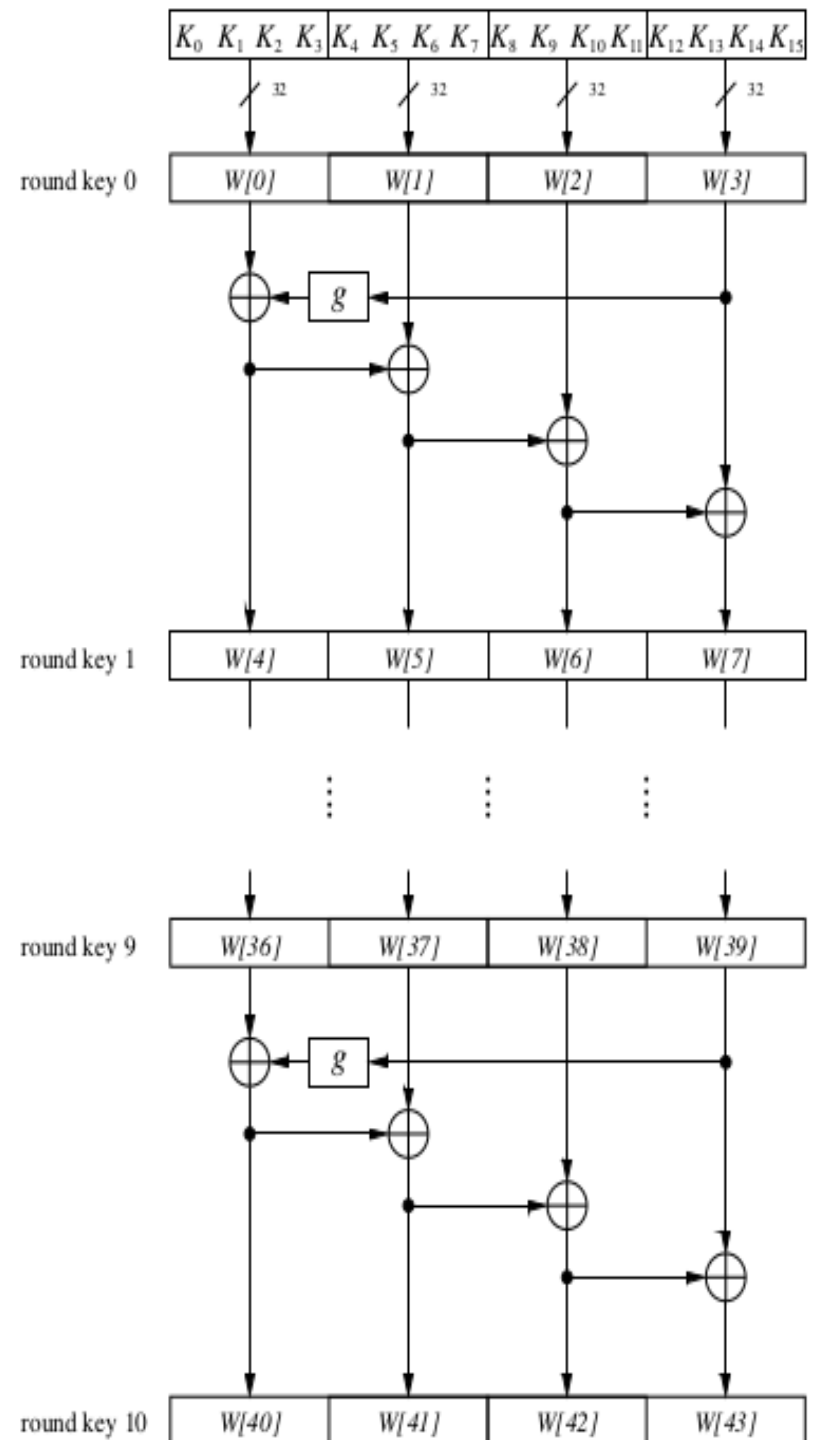
(an XOR addition of a subkey is used both at the input and output of AES. This process is sometimes referred to as **key whitening**.)

The number of subkeys is equal to the number of rounds plus one, due to the key needed for key whitening in the first key addition layer.

For the key length of 128 bits, the number of rounds is $nr = 10$, and there are 11 subkeys, each of 128 bits.

The AES key schedule is word-oriented, where 1 word = 32 bits. Subkeys are stored in a key expansion array W that consists of words.

There are different key schedules for the three different AES key sizes of 128, 192 and 256 bit, which are all similar.



Key Scheduling (contd.)

The first subkey k_0 is the original AES key, i.e., the key is copied into the first four elements of the key array W . The other array elements are computed as follows. As can be seen in the figure, the leftmost word of a subkey $W[4i]$, where $i = 1, \dots, 10$, is computed as:

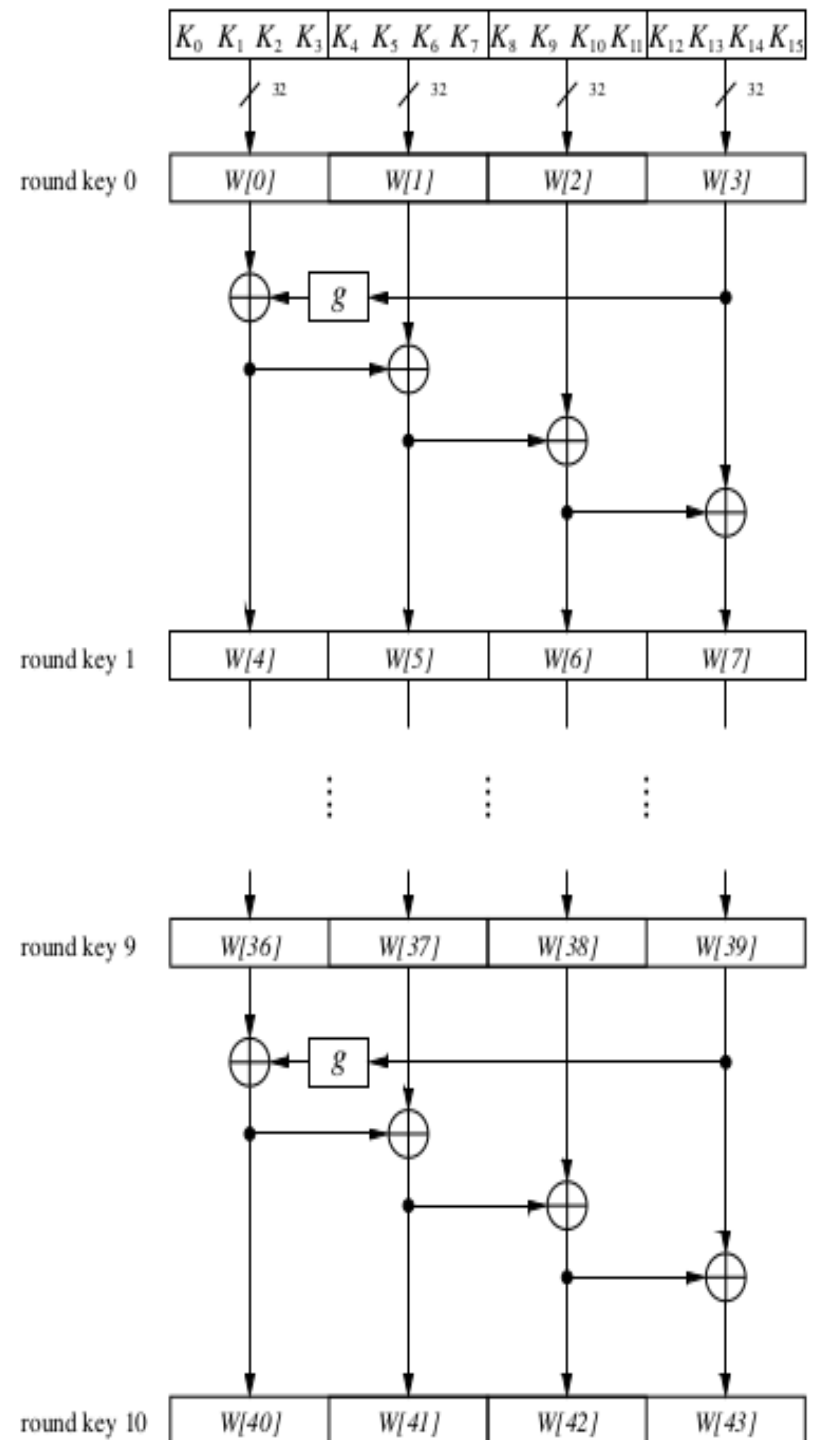
$$W[4i] = W[4(i-1)] + g(W[4i-1])$$

Here $g()$ is a nonlinear function with a four-byte input and output. The remaining three words of a subkey are computed recursively as:

$$W[4i+j] = W[4i+j-1] + W[4(i-1)+j]$$

where $i = 1, \dots, 10$ and $j = 1, 2, 3$.

The function $g()$ rotates its four input bytes, performs a byte-wise S-Box substitution, and adds a round coefficient RC to it.



$g()$ and RC

The round coefficient is an element of the Galois field $GF(2^8)$, i.e, an 8-bit value. It is only added to the leftmost byte in the function $g()$. The round coefficients vary from round to round according to the following rule:

$$RC[1] = x^0 = (0000\ 0001)_2,$$

$$RC[2] = x^1 = (0000\ 0010)_2,$$

$$RC[3] = x^2 = (0000\ 0100)_2,$$

...

$$RC[10] = x^9 = (0011\ 0110)_2.$$

The function $g()$ has two purposes. First, it adds nonlinearity to the key schedule. Second, it removes symmetry in AES.

